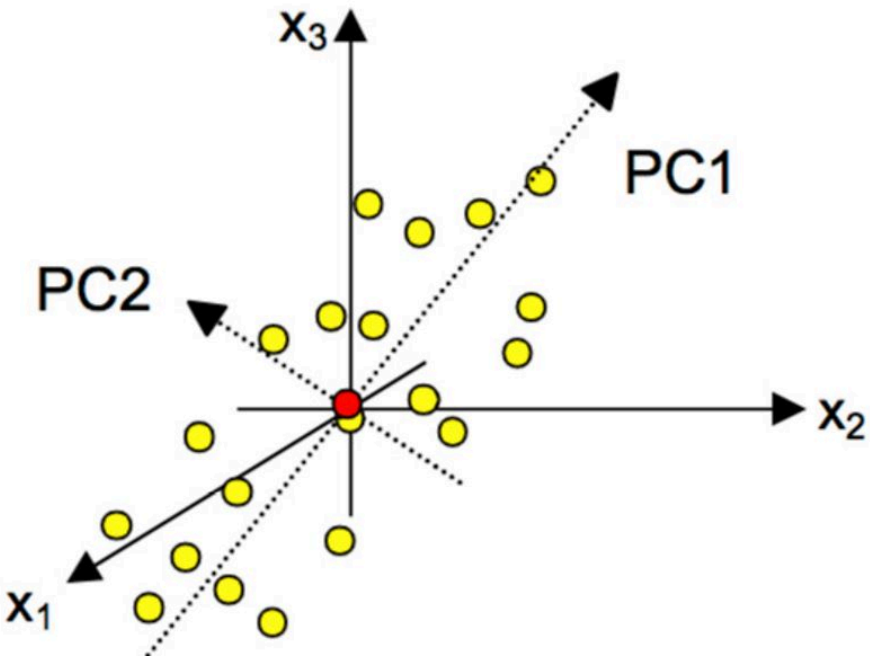
ICLR: IN-CONEXT LEARNING OF REPRESENTATIONS

- **Background Concepts**
  - Principal Component Analysis: a way of **dimensionality reduction** in order to visualize the high-dimensional data by linearly transforming the data "onto a new (mostly low level) coordinate system such that directions (**principle components**) capturing the largest variation in the data can be easily identified" .
  - 1st Principle Component: the first principal component (PC1) is the line that best accounts for the shape of the point swarm. It represents the maximum variance direction in the data. Each observation (yellow dot) may be projected onto this line in order to get a coordinate value along the PC-line, as shown in the figure below. To calculate the 1st Principle Component:

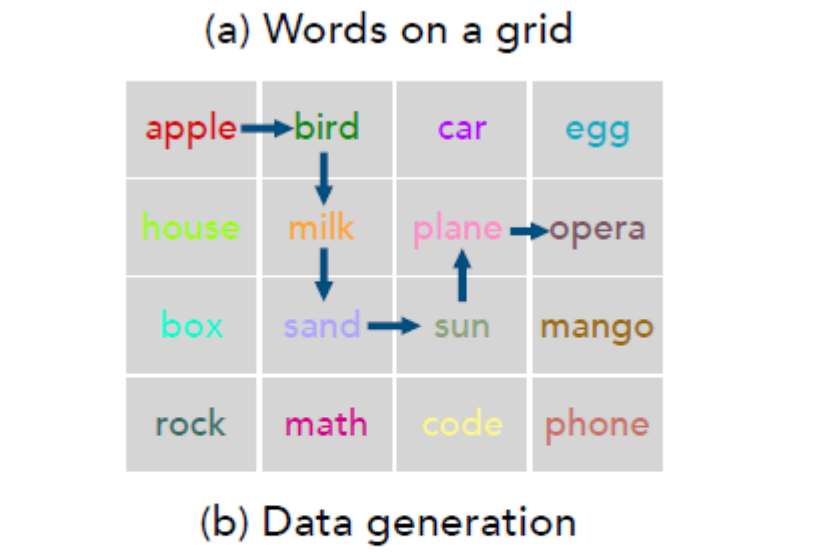
$$\text{where covariance matrix} = \begin{bmatrix} \mathrm{Cov}(X_1, X_1) & \mathrm{Cov}(X_1, X_2) & \dots & \mathrm{Cov}(X_1, X_n) \\ \mathrm{Cov}(X_2, X_1) & \mathrm{Cov}(X_2, X_2) & \dots & \mathrm{Cov}(X_2, X_n) \\ \vdots & \vdots & \ddots & \vdots \\ \mathrm{Cov}(X_n, X_1) & \mathrm{Cov}(X_n, X_2) & \dots & \mathrm{Cov}(X_n, X_n) \end{bmatrix}$$

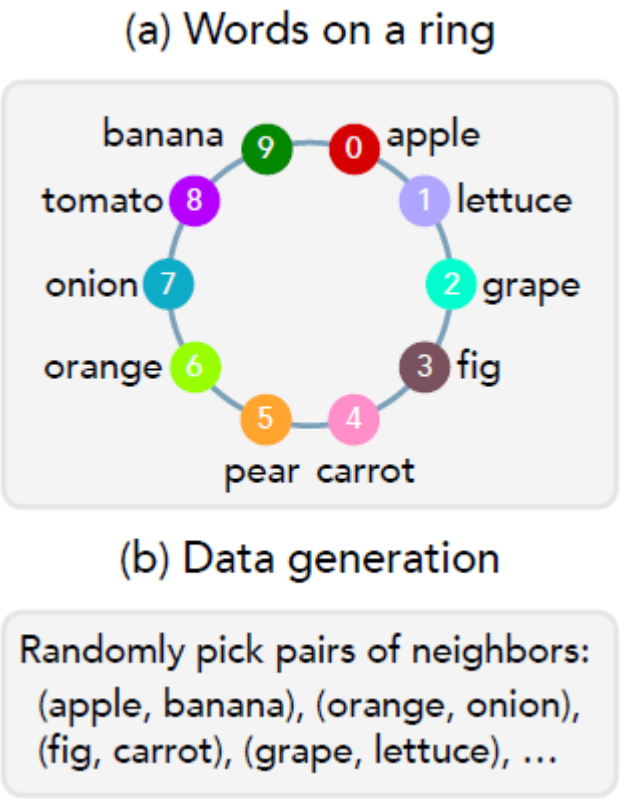
  - Further Principle Components: The i-th principle component is orthongonal to the (i-1)-th principle component.



  - In-Context Learning: the ability of LLMs to learn and adapt to new tasks or patterns by simply seeing a few examples within the **prompt**, with no need to fine-tune (*For example, assume that we describe to an LLM that a new product called strawberry has been announced. Ideally, based on this context, the model would alter the representation of strawberry and reflect that we are not referring to pretraining semantics (e.g., the fruit strawberry)*).
  - Representation in LLMs: LLMs encode knowledge about the world in the form of internal numerical representations associated with each token (the dynamic, or in-process embeddings).
- **Experiment**

- Find a collection of words, preferably the words **commonly seen** (so that the model can learn from the pre-train stage) but have a **low relationship** between each two (for example, bird, car, sand, sodium, ...).

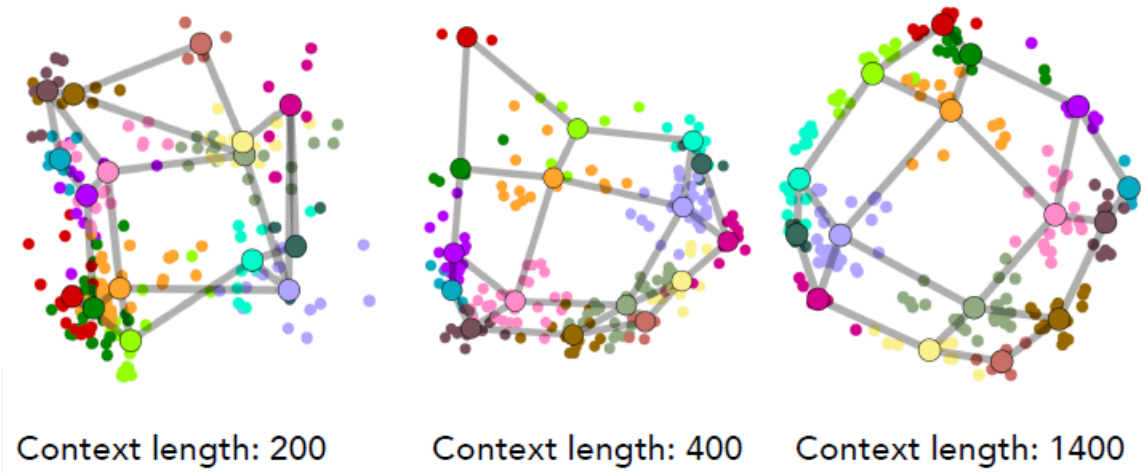




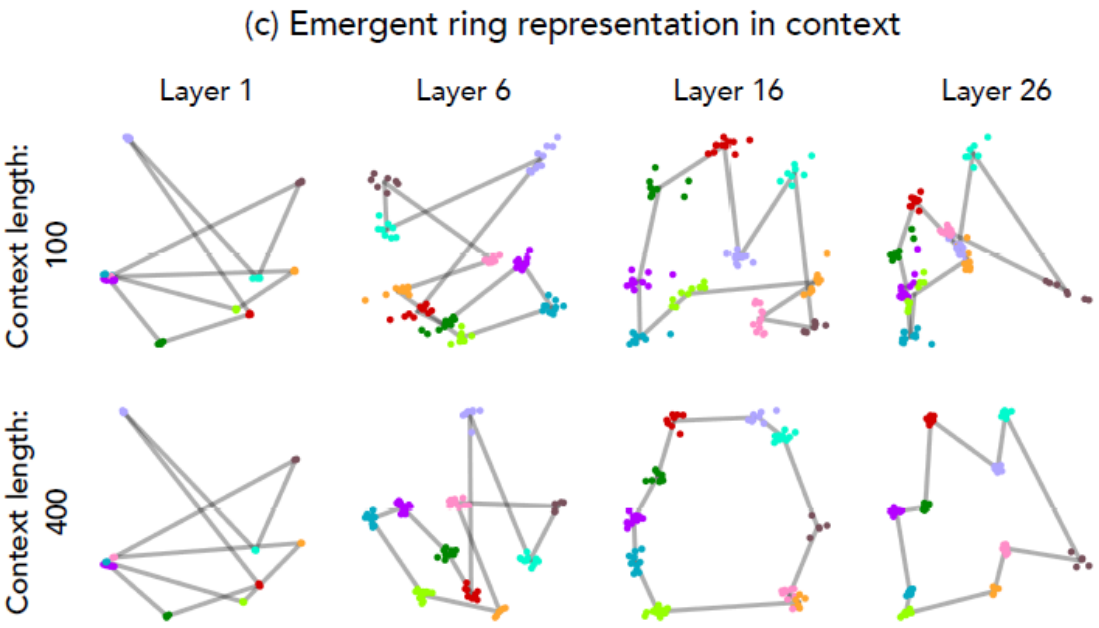
- Then, place the words as nodes of a graph (grid, ring, etc) where the neighbor has a strong, new, in-context relationship.
- **Randomly** walk through the graph to generate context and feed the model.

- At each timestamp, on a layer l, peek at the activations of $N_w$input tokens; then compute the mean for each token, denote the mean of the representation of each token as $h^l_{\tau_i}$, where token $\tau_i \in \mathrm{T}$. Stack the mean-representation of the input tokens on layer $l$as $H^l(\mathrm{T})$. Then run the PCA on the stacked matrix and select the 1st and the 2nd principal components to visualize model activations.

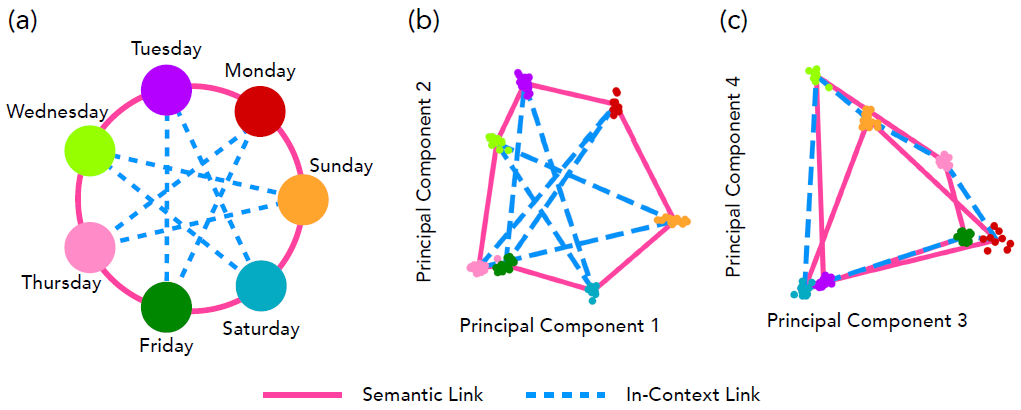
- **Results**
  - The grid example shows that, on $l = 26$, the number of in-context exemplars increases, there is a formation of representations mirroring the grid structure underlying the data-generating process.



    - Here we can clearly see that when we feed short contexts to the model, it will still follow the pattern (semantic relationship) it learned from the pre-train stage: the pink dots (plane) and the purple dots (car) are very close to each other (they are both vehicles); the violet dots (sand) and the brown dots (mango) are very far.
    - However, when the context length increases to 1400, the distribution of dots looks similar to the grid presented above, meaning that the model has learned in-context relationships and given up semantic relationships.
  - The ring example shows that, as the number of in-context exemplars increases, there is a formation of representations mirroring the ring structure underlying the data-generating process. Also, a deeper layer can often learn the in-context relationship better.



- Construct a heptagon with the name of days in a week as nodes. The edges of the heptagon are the semantic relationships (i.e. Counterclockwise traverse gives the order of a week), and the internal dashlines are the in-context relationships. Repeating the procedures gives:



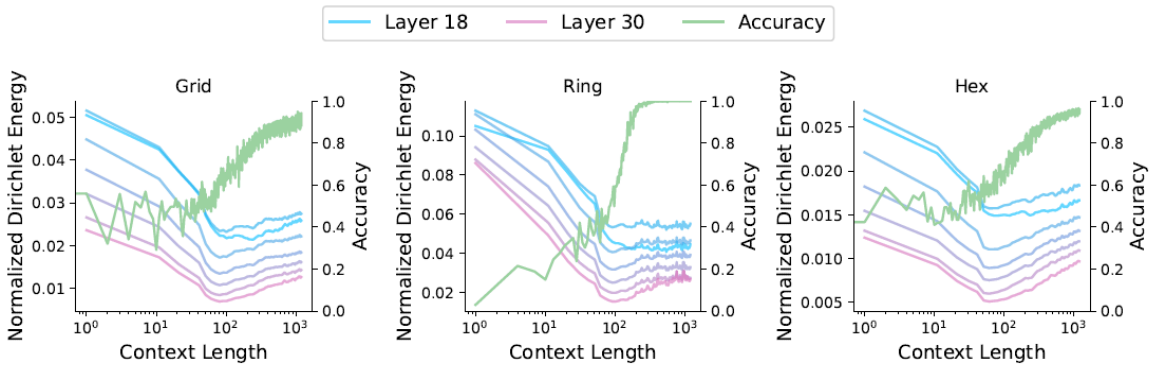
  - We can see that on the 1st principal component, where the relationship is closer to the semantic relationship (i.e. Tuesday is close to Monday and Wednesday), the in-context relationship is not clear (Monday is still far from Thursday and Friday, but we want them to be close).
  - On the other hand, for the 3rd PC, where the in-context relationship is clear, the original semantic relationship is vague.
  - We can conclude that when there is a conflict between semantic link and in-context link, the first two (according to the paper) PCs will still show semantic link, but the following PCs will show the in-context relationship, meaning that the latter is not dominant. However, when there are enough contexts, it's shown that the model **overrides** the semantic prior to perform well.

- Another way to quantitively gauge performance (*"how aligned the representations are with the structure of the graph that underlies the context"*) is using a *simplified* version of **Dirichlet Energy** (classically, this gauge is to measure the smoothness of functions/signals).

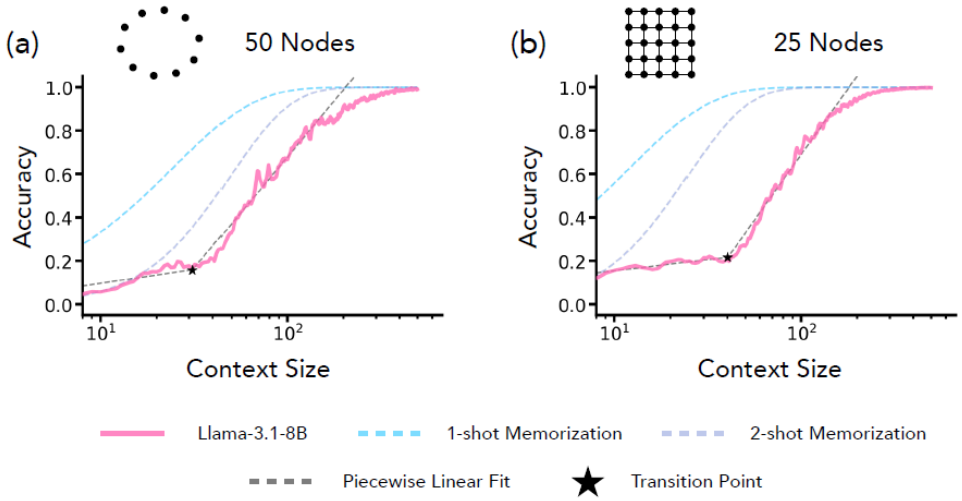
$$E_G(\mathbf{H}^l(\mathrm{T})) = \sum_{i,j} \mathbf{A}_{i,j} ||h_i^l - h_j^l||^2, \text{where}$$

$A_{i,j}$ is the element of Adjacency Matrix (if i and j are connected then $\mathrm{A[i][j]} = 1$ otherwise 0)

Among pairs of **neighbors** in in-context relationships, we calculate the square of their Euclid distance. The better the model learns the in-context relationships, the lower E_G should be (as the neighbors are closer to each other). The relationships of E_G vs. context length are given below:



- It can be found that $E_G$truly **decreases** with context length; the deeper the layer is, the smaller $E_G$will be; and the accuracy is opposite to the pattern of E_G.
- It's also worth noting that, with some length of context, the performance of the model will suddenly increase rapidly, just like the **emergent** phenomenon. *"This leads us to the claim that as the amount of context is scaled, there is an emergent re-organization of representations that allows the model to perform well on our in-context graph tracing task. We note these results also provide a more quantitative counterpart to our PCA visualization results before."*
- Can this be a trivial phenomenon? Can this be simply a **"memorization solution"**, where the model merely regurgitating a node's neighbors by copying them from its context?
  - Assume the model can learn the nodes (tokens) from the context by one-shot memorization (learn by at least seeing once), then given a token in the context, the probability that the model **doesn't** learn a specific node (token)x from $n$nodes is $\frac{n-1}{n}$. Assume the length of the context is l, then the probability for the whole context will be $(\frac{n-1}{n})^l$. The probability that the model does learn token $x$is $p(x) = 1 - (\frac{n-1}{n})^l$。
  - Without losing generality, we can consider the second-shot situation. The probability that the model exactly sees node $x$once is $l(\frac{1}{n})(\frac{n-1}{n})^{l-1}$ ($\frac{1}{n}$is for seeing exactly once, $(\frac{n-1}{n})^{l-1}$is for "not seen in other $l-1$tokens, and multiply $l$because the token $x$can be anywhere). Then the probability of second-shot is $p_2(x) = p(x) - l(\frac{1}{n})(\frac{n-1}{n})^{l-1}$.
  - However, the plots of these two cases are smooth, while the actual accuracy plot contains an obvious transition point, as shown below:



- Thus, this memorization model cannot describe the pattern of increment of accuracy.

- **Energy minimizing theory**

  (In this section, h_i will be replaced by z_i to represent the value of nodes)

  - One way to demonstrate the scenario presented above is the model implicitly runs an internal **energy minimizer** to find a solution $\mathbf{z}$ such that $E_G(\mathbf{z})$ is minimized.
  - We call k-th energy minimizer $\mathbf{z}^{(k)}$ an optimal solution **iff** it minimizes E_G and **orthogonal** (as we don't want each solution to be dependent on one another) to the previous k-1 solutions. Formally, the solution to the problem:

    $$z^{(k)} = \arg\min_{z \in S^{n-1}} E_G(z) \quad \text{s.t.}$$

    $$z \perp z^{(j)}, \quad \forall j \leq k-1.$$

    is $\{\mathbf{z}^{(k)}\}_{k=1}^{n}$.
  - There exists a trivial solution to this problem, denoted as $\mathbf{z}^{(1)}$, in which all values for nodes in the graph are a non-zero constant (so that $||z_i - z_j||^2$ is always 0). The degeneration will lose all details about the graph.
  - According to Spielman, we can use $(z_i^{(2)}, z_i^{(3)})$, a good planar embedding, to represent node i, called **spectral embeddings** (meaning that we can use the embedding to recover the original graph on a 2-D plane). It is shown (in Appendix B of the paper) that if the model runs an energy minimizing mechanism to minimize E_G, and the solution isn't degenerated (trivial solution), then the first two PC's will exactly produce spectral embedding $(\mathbf{z}^{(2)}, \mathbf{z}^{(3)})$.
  - Special Situation: **what if the token graph is not connected?**
    - Given a token graph $\hat{G}$ with q**separate** connected components, we want to apply the energy minimizing theory on it. We can construct the first q energy minimizers by:

      $$z_j^{(i)} = \begin{cases} -\alpha_i, & \text{if } j \in \bigcup_{k=1}^{i-1} U_k, \\ 1, & \text{else.} \end{cases} \text{, where}$$

      $$U_i \text{ denotes the set of nodes in the i-th component}$$

      $$\alpha_1 = 1,$$

      $$\alpha_i = \frac{\sum_{k'=i}^{q} \sum_{j \in U_{k'}} z_j^{(i-1)}}{\sum_{k=1}^{i-1} \sum_{j \in U_k} z_j^{(i-1)}}, \quad \text{for } i \in [q] \setminus \{1\} \text{ to fulfill the orthogonality}$$

        - For example: given a graph with 3 ($q = 3$) connected components, with 2 nodes in each component, we can have:
            - $\mathbf{z}^{(1)} = [1, 1, 1, 1, 1, 1]$ (the trivial solution)
            - $\mathbf{z}^{(2)} = [-\alpha_2, -\alpha_2, 1, 1, 1, 1]$, only the 1st component will have a value of $-\alpha_2$as $j \in \bigcup_{k=1}^{i-1} U_k, i = 2 \rightarrow U_1$
            - $\mathbf{z}^{(3)} = [-\alpha_3, -\alpha_3, -\alpha_3, -\alpha_3, 1, 1]$, as $j \in \bigcup_{k=1}^{i-1} U_k, i = 3 \rightarrow U_1 \cup U_2$
        - It's easy to see that in each component, $E_G$is 0, so the non-connected minimizer given above is global. Also, we can conclude that *"if the graph is unconnected, then the energy-minimizing representations will be dominated by information-less principal components, in which we should not expect any meaningful visualization";* it can only keep the **connectivity** information (i.e. different values in $\mathbf{z}^{(i)}$can only show some nodes are in the same group) but will lose the structural info (you cannot recover the original graph structure from ONLY the $\mathbf{z}^{(i)}$vector).
        - And because of the conclusion above, we, therefore, can have a key takeaway about the material for in-context learning: **When designing in-context training tasks, it is best to use a fully connected token graph (or ensure that the graph has a single large connected component). This can help the model more effectively extract task-relevant structural information from the context, leading to better in-context learning performance.**
        - One possible way to explain the **in-context emergent** is the problem of bond-percolation on a graph, where one starts from the unconnected graph and gradually fills the edge to connect the nodes; after that, there will be a *second-order transition*, resulting in a large connected component.
- **How to explain in-context learning by the structure of transformers?**
    - The answer might lie in the **Residual Connection:** $x^{l+1} = F(x^l) + x^l$**.**
    - As mentioned before, we add the result of the current layer to the result of the previous layer to form the input of the latter layer. This can truly help handle gradient disappearance and increase the robustness of the training process, but this can also **preserve the information** from the input and learned from previous layers, making it easier to learn and memorize.
    - Each layer is only responsible for changing part of the representation to correct or refine the answer (*not modified, but added)*, which is why it is important to track where the information is at a given layer.
    - Knowledge is accumulative in transformers. The deeper the layer is, the more abstract and complicated the captured knowledge will be.
    - Because of the "gradual addition" property of the residual stream, in-context learning, which is **overriding** the old knowledge with the new one, is possible and low-cost.

- **Work Cited**
    - Paper
    - NEW: Better In-Context Learning ICL, Improved RAG (Harvard)
    - Dirichlet Energy Minimization Explains In-Context Learning (Harvard)
    - Principal component analysis

- What Is Principal Component Analysis (PCA) and How It Is Used?